

KE Pipeline

Minimal Walkthrough

How Agent-X computes $\frac{1}{2}mv^2$

*A step-by-step trace through every tantra,
with the minimal set of nodes defined.*

Question: “mass is 5 kg and velocity is 10 m/s. find kinetic energy”
Answer: kinetic-energy = 250

Generated from pathram2 composable sections
doc-36 (ke-walkthrough) with 14 linked sections

Table of Contents

1. The Question
2. What is a Triple
3. The Graph and the Tensor
4. Mapping, Not Matching
5. The Minimal Node Set
6. Step 1: build-question-graph
7. Step 2: assertion-bandha
8. Step 3: avrti-refine-v2
9. Steps 4–5: grade-split and count-chain
10. Step 6: kosha-expand (PPR)
11. Step 7: detect-signals
12. Step 8: dispatch-answer (derive path)
13. The Complete Triple History
14. Connecting to the Math Foundations

1. The Question

The input is a single string:

"mass is 5 kg and velocity is 10 m/s. find kinetic energy"

The expected answer is: kinetic-energy = 250

This is $\frac{1}{2} \times 5 \times 10^2 = \frac{1}{2} \times 5 \times 100 = 250$ joule.

The formula is: $KE = \frac{1}{2}mv^2$

The question has two sentences (separated by a period): - Sentence 1: "mass is 5 kg and velocity is 10 m/s" (data) - Sentence 2: "find kinetic energy" (intent)

The pipeline must: parse the words, bind numbers to concepts, find the right formula, compute, and return the answer. Every step works by adding triples to a growing graph.

2. What is a Triple

A triple is the atom of knowledge in Agent-X. Everything the system knows, discovers, or computes is expressed as a triple.

The Shape: (Subject, Edge, Object)

A triple is a 3-tuple: (subject, edge, object). Three parts, always three, no more, no less.

- **Subject**: the thing we are talking about - **Edge**: the relationship (what kind of connection) - **Object**: what it connects to

Examples from the KE question:

Subject	Edge	Object	In English
mass	satya	mass	"mass exists in the graph"
mass	sankhya	5.	"mass has the value 5"
mass	matra	kilogram	"mass is measured in kilogram"
find	vidhi-kaala	solve-for	"find is an intent to solve"

Why Triples?

A triple is the simplest structure that carries meaning. Two parts (a pair) can only say "A relates to B" but not how. Four or more parts can always be decomposed into triples.

A triple maps directly to one cell in the 3D tensor:

$$(\text{subject}, \text{edge}, \text{object}) = G[\text{subject}][\text{object}][\text{edge}] = 1$$

The tensor $G \in \{0, 1\}^{N \times N \times R}$ has three axes: N nodes, N nodes, R relation types. Each triple sets exactly one cell from 0 to 1.

The Two Kinds of Triples

Permanent triples: edges in the knowledge graph, loaded from .om files. These exist before any question is asked. Example: $G[\text{kinetic-energy-mantra}][\text{mass}][\text{janya}] = 1$ (mass is an input to the KE formula). These are the 1s that are always there.

Question triples: created during the pipeline for this specific question. Example: $[\text{mass}, \text{sankhya}, 5.]$ — mass has value 5 in THIS question. These are temporary 1s that exist only during processing.

Monotonicity: Triples Only Accumulate

Once a triple is added, it is never removed. The pipeline can only set 0s to 1s, never 1s to 0s. This is the monotonicity property of the boolean ring: $a \oplus b \geq a$.

After each pipeline step, the triple count is the same or larger. The final answer is read from the accumulated triples.

Reading the Triple Table

Throughout this document, we show triples as tables. Each row is one triple. The columns are always: Subject, Edge, Object. The edge tells you which floor (dimension) of the tensor this triple lives on.

3. The Graph and the Tensor

Before the pipeline runs, there is a permanent knowledge graph. This section defines what it is and how triples relate to it.

The 3D Tensor

The graph is a 3D array of 0s and 1s:

$$G \in \{0, 1\}^{N \times N \times R}$$

- N = number of nodes (currently 1604) - R = number of relation types (currently 62) - $G[i][j][r] = 1$ means: node i is connected to node j by relation r

Each relation type is a "floor" — a 2D slice of the tensor. Floor 0 might be swarupa, floor 1 might be yukta, etc. Each floor is an $N \times N$ matrix of 0s and 1s.

A Node is a Vertical Slice

A node is not just a name. It is the pattern of all 1s in its row and column across all 62 floors.

Node "mass" means: look at row "mass" across all 62 floors. Every 1 in that row is an edge FROM mass. Every 1 in column "mass" (across all floors) is an edge TO mass. Together, these 1s define what "mass" is.

mass has 1s on: - yukta floor: connected to kinetic-energy, momentum, kilogram, ... - matra floor: connected to kilogram, metre, second - phala floor: connected to kinetic-energy-mantra, momentum-mantra, ... (formulas mass participates in) - vishesa floor: connected to linear-force-varga - swarupa floor: connected to subanta, force

Without these edges, "mass" is just a string. With them, it is a concept with relationships, units, formulas, and context.

Edges Are Typed

Each edge lives on exactly one floor. The floor number IS the relation type. This is what makes triples work: (mass, yukta, kinetic-energy) sets $G[\text{mass}][\text{kinetic-energy}][\text{yukta}] = 1$.

The 10 core relation types (visheshanam):

Relation	What it means	Example
swarupa	identity / is-a	kinetic-energy swarupa energy
abheda	similar-to	kinetic-energy abheda work
yukta	compatible-with	mass yukta kinetic-energy
sthita	participates-in	velocity sthita velocity-step
siddha	proven-by	kinetic-energy siddha pbr
kriya	operates-on	(used in ring multiplication)
phala	result-of	kinetic-energy-mantra phala kinetic-energy
janya	input-to	kinetic-energy-mantra janya mass
drishthanta	example-of	velocity drishthanta uttara-avastha
pratipaksha	opposite-of	kinetic-energy pratipaksha potential-energy

Plus 52 dynamic dimensions (matra, sankhya, kaala, vachana, ...) that carry metadata.

janya and phala: Functions, Not Direct Outputs

This is important: mass does NOT directly produce kinetic-energy. The edges are:

kinetic-energy-mantra --janya--> mass (mass is an input)
 kinetic-energy-mantra --janya--> velocity (velocity is an input)
 kinetic-energy-mantra --phala--> kinetic-energy (KE is the output)

The mantra is a function: $f(\text{mass}, \text{velocity}) \rightarrow \text{kinetic-energy}$. The phala edge goes from the FUNCTION to its output, not from any single input.

mass also has phala edges, but they point to MANTRAS (functions it participates in), not to results:

mass --phala--> kinetic-energy-mantra (mass is used in the KE formula)
 mass --phala--> momentum-mantra (mass is used in the momentum formula)
 mass --phala--> potential-energy-mantra (mass is used in the PE formula)

This means: mass alone produces nothing. It is an ingredient. Only when ALL janya inputs of a mantra

have values does the mantra fire. This is the $\otimes$ (AND) operation — every input must be present simultaneously.

For the KE question: mass=5 AND velocity=10 $\rightarrow$ both janya inputs present $\rightarrow$ kinetic-energy-mantra fires $\rightarrow$ phala = kinetic-energy = 250.

If only mass=5 were given (no velocity), the mantra would NOT fire. No answer.

Permanent vs Question Graph

The permanent graph has ~41000 edges across 1604 nodes. These are loaded once from .om and .shabda files.

When a question arrives, the pipeline creates NEW triples that sit alongside the permanent ones. These question triples use the same format (subject, edge, object) and the same floors. The pipeline reads BOTH permanent and question triples to make decisions.

After the answer is returned, the question triples are discarded. The permanent graph is unchanged.

4. Mapping, Not Matching

The pipeline does NOT do string matching. It does not use regex, substring search, or keyword lists. Instead, every word lookup is a mapping — a morphism from word-space to node-space.

The Difference

String matching (what Agent-X does NOT do): scan the input for patterns like "mass", check if the string contains "kg", use regex to extract numbers. This is brittle — it breaks when words change, when languages change, when new concepts are added.

Mapping (what Agent-X does): each word is mapped to a graph node through a named edge (naama). The mapping is stored in the graph itself, not in code.

How shabda-anveshana Works

shabda-anveshana ("word-search") is the mapping function. For a given word:

1. Look up the word in the shabda index (built from word: and alias: entries in .shabda files)
2. The shabda entry points to a graph node
3. Return that node (or nothing, if the word is unknown)

The mapping lives in the graph:

shabda entry	word: value	maps to node
kilogram	kg, kilogram	kilogram
metre-per-second	m/s	metre-per-second
copula	is, are, be, was, were	copula
vidhi-kaala	find, compute, calculate, determine, solve	vidhi-kaala
mass	(naama edge)	mass
velocity	(naama edge)	velocity
kinetic-energy	(naama edge)	kinetic-energy

This is a function $f : \text{Words} \rightarrow \text{Nodes} \cup \{\emptyset\}$. It is a morphism — it preserves structure. The word "kg" does not contain the string "mass" anywhere, but the mapping knows that $\text{kg} \rightarrow \text{kilogram} \rightarrow \text{mass}$ (via concepts-for-unit).

Why This Matters

1. **No hardcoded word lists in code:** all word mappings are in .shabda files and .om naama edges. Adding a new word means adding a shabda entry, not changing code.
2. **Composable:** the mapping chains. "kg" $\rightarrow$ kilogram (word lookup) $\rightarrow$ mass (concepts-for-unit). Each step is a separate mapping.
3. **Language-independent:** the same graph node "mass" could be mapped from "kg" (English), "kilo-gramm" (German), or any other word. The pipeline code never changes.
4. **Unit binding is a mapping:** when "5 kg" appears, the pipeline does not parse "kg" as a unit string. It maps "kg" to the node kilogram, reads concepts-for-unit: mass from kilogram's shabda, and emits [mass, sankhya, 5.] and [mass, matra, kilogram]. The number 5 gets bound to mass through a chain of graph lookups, not string operations.

The Two-Step Chain for Units

```

"5 kg"
  |
  v
split-numeric: num="5", unit-str="kg"
  |           |
  v           v
[5, asprista-sankhya, 5.] shabda-anveshana("kg") = kilogram
                          |
                          v
                      concepts-for-unit: mass
                          |
                          v
                    [mass, sankhya, 5.] + [mass, matra, kilogram]

```

Every arrow is a mapping. No string matching anywhere.

5. The Minimal Node Set

To answer this question, the pipeline touches exactly these nodes in the permanent graph. Each node is a row in the 3D tensor $G \in \{0, 1\}^{N \times N \times R}$.

Concept Nodes

Node	What it is	Key edges (floor = 1)
mass	a physical quantity	vishesa → linear-force-varga, matra → kilogram, phala → kinetic-energy-mantra (participates in formula)
velocity	a physical quantity	vishesa → linear-motion-varga, matra → metre-per-second, phala → kinetic-energy-mantra (participates in formula)
kinetic-energy	a physical quantity	vishesa → mechanical-energy-varga, pratipaksha → potential-energy

These are the concepts the question talks about. Each lives in the kosha layer. Note: mass and velocity have phala edges to the mantra (the function they participate in), NOT directly to kinetic-energy.

Unit Nodes

Node	shabda word:	shabda concepts-for-unit:	What it does
kilogram	kg, kilogram	mass, link-mass	When "kg" is mapped, bind the preceding number to mass
metre-per-second	m/s	velocity	When "m/s" is mapped, bind the preceding number to velocity

Units are the bridge between human text ("5 kg") and graph concepts (mass = 5). The binding is a mapping chain: word → unit node → concept (via concepts-for-unit).

The Formula Node (Mantra)

Node	shabda	Edges
kinetic-energy-mantra	name: kinetic-energy, unit: joule	janya → mass, janya → velocity, phala → kinetic-energy

The mantra is a function: $f(\text{mass}, \text{velocity}) \rightarrow \text{kinetic-energy}$.

- janya edges point to the INPUTS: mass and velocity - phala edge points to the OUTPUT: kinetic-energy
- The mantra only fires when ALL janya inputs have bound values

The edges form a triangle:

mass —phala→ kinetic-energy-mantra —phala→ kinetic-energy
velocity —phala→ kinetic-energy-mantra

Reading this: mass and velocity both contribute (phala) to kinetic-energy-mantra. The mantra reads them (janya) and produces (phala) kinetic-energy. Neither mass alone nor velocity alone can produce kinetic-energy.

The mantra also has krama (sequence) edges that encode the s-expression:

(half (multiplication mass (square velocity)))

This reads: take velocity, square it, multiply by mass, halve the result.

Operation Nodes

Node	eval:	arity:	What it computes
half	half	1	$x \mapsto x \times 0.5$
multiplication	mul	2	$(x, y) \mapsto x \times y$
square	square	1	$x \mapsto x^2$

These are pure functions. Their shabda carries the eval: key (the OCaml primitive name) and arity: (how many inputs).

Grammar Nodes

Node	shabda word:	Edge emitted	Role
copula	is, are, be, was, were	[word, copula, word]	Links subject to predicate
dvandva	and	[word, dvandva, dvandva]	Separates clauses (grade boundary)
vidhi-kaala	find, compute, calculate	[word, vidhi-kaala, solve-for]	Intent signal: "I want to find X"

These are also mapped, not matched. "is" maps to copula because copula has word: is in its shabda. "find" maps to vidhi-kaala because vidhi-kaala has word: find. The mapping lives in the graph, not in code.

Dimensions (Floors) Used

The 3D tensor has 62 floors. This question only uses a handful:

Floor	Name	What it records
satya	truth	"this word was mapped to a graph node"
asprista-sankhya	untouched number	"a raw number, not yet bound"
sankhya	bound number	"number X is bound to concept Y"
matra	unit	"concept Y is measured in unit Z"
copula	linking verb	"is" connecting subject to predicate
dvandva	conjunction	"and" separating clauses
viraam	period/stop	sentence boundary
vidhi-kaala	imperative	"find" = intent to solve
swarupa	identity	X IS Y (from assertion-bandha)
kaala	tense	verb tense
vachana	number	singular/plural
janya	input-of	"mass is an input to kinetic-energy-mantra" (permanent edge)
phala	result-of	"kinetic-energy-mantra produces kinetic-energy" (permanent edge)
krama	sequence	"the formula steps in order" (permanent edge)
kosha-janya	PPR-expanded	"nearby concept found by graph walk"

That is 15 out of 62 floors. The rest are silent (all 0s) for this question.

Note: janya, phala, and krama are permanent graph edges (loaded from .om files). The question pipeline reads them but does not create them. The pipeline creates triples on satya, sankhya, matra, copula, dvandva, viraam, vidhi-kaala, swarupa, kaala, vachana, and kosha-janya floors.

6. Step 1: build-question-graph

What it does

Takes the raw sentence string and maps each word to graph nodes. No string matching — every lookup is a mapping through the graph.

For each word, left to right: 1. **Map** the word via shabda-anveshana (naama edge lookup in the graph) 2. If mapped to a concept node: emit [concept, satya, concept] 3. If it is a number: emit [number, asprista-sankhya, number] 4. If the word maps to a unit node: read concepts-for-unit from its shabda, emit [concept, sankhya, value] and [concept, matra, unit-node] 5. If it maps to a grammar node: emit the grammar signal triple

Important: step 1 is not string matching. The word "kg" does not contain "mass". But the mapping kg → kilogram → mass (via concepts-for-unit) connects them through graph edges.

The input

"mass is 5 kg and velocity is 10 m/s. find kinetic energy"

Words processed left to right: mass, is, 5, kg, and, velocity, is, 10, m/s, ., find, kinetic energy

Note: "kinetic energy" is two words but the compound pre-pass merges them into "kinetic-energy" because the graph has a compound node with that name. This merge is also a mapping — it checks if consecutive words form a known compound by looking up the graph, not by string matching.

The mapping chain for each word

Word	Mapping	Result
mass	shabda-anveshana("mass") = mass	concept node found
is	shabda-anveshana("is") = copula (via word: entry)	grammar node found
5	split-numeric: num=5, unit=""	number detected
kg	shabda-anveshana("kg") = kilogram (via word: entry)	unit node found
and	shabda-anveshana("and") = dvandva	grammar node found
velocity	shabda-anveshana("velocity") = velocity	concept node found
is	shabda-anveshana("is") = copula	grammar node (again)
10	split-numeric: num=10, unit=""	number detected
m/s	shabda-anveshana("m/s") = metre-per-second (via word: entry)	unit node found
.	punctuation detected	viraam signal
find	shabda-anveshana("find") = vidhi-kaala (via word: entry)	intent node found
kinetic-energy	shabda-anveshana("kinetic-energy") = kinetic-energy	concept node found

Every arrow above is a graph lookup. None is a string operation.

Output: 14 triples

#	Subject	Edge	Object	Floor	How it was produced
1	mass	satya	mass	satya	mapped "mass" to node, emit existence
2	is	copula	is	copula	mapped "is" to copula node
3	5	asprista-sankhya	5.	asprista-sankhya	numeric, no unit yet
4	mass	sankhya	5.	sankhya	"kg" mapped to kilogram, concepts-for-unit = mass, bind 5 to mass
5	mass	matra	kilogram	matra	unit annotation from same mapping
6	and	dvandva	dvandva	dvandva	mapped "and" to dvandva node
7	velocity	satya	velocity	satya	mapped "velocity" to node
8	is	copula	is	copula	second "is" mapped again
9	10	asprista-sankhya	10.	asprista-sankhya	numeric
10	velocity	sankhya	10.	sankhya	"m/s" mapped to metre-per-second, concepts-for-unit = velocity
11	velocity	matra	metre-per-second	matra	unit annotation
12	.	viraam	.	viraam	period = sentence boundary
13	find	vidhi-kaala	solve-for	vidhi-kaala	mapped "find" to intent node
14	kinetic-energy	satya	kinetic-energy	satya	compound mapped to node

The unit binding in detail

When "5 kg" is processed:

1. "5" is a number → emit [5, asprista-sankhya, 5.] (raw number, not bound yet) 2. "kg" maps to kilogram via shabda-anveshana 3. kilogram has shabda: concepts-for-unit = mass 4. The previous unbound number (5) gets bound to mass: [mass, sankhya, 5.] 5. The unit is annotated: [mass, matra, kilogram]

Three triples from two words. Each step is a mapping through the graph, not a string operation.

What this step is in math terms

build-question-graph is a morphism: String → Set of triples. It is a composition of smaller morphisms: split → map-each-word → emit-triples. Each triple sets one cell in the 3D tensor from 0 to 1. This step activates 14 cells.

7. Step 2: assertion-bandha

What it does

Scans for subject-copula-object patterns within each grade (sentence). When it finds "X is Y" where both X and Y are graph nodes, it emits [X, swarupa, Y].

Before (14 triples from Step 1)

Pattern found

In sentence 1: mass ... is ... velocity. But wait — "is" sits between "mass" (with its value) and "velocity" (next entity). The tantra sees: - mass (satya) → is (copula) → ... → velocity (satya after copula)

So it emits: [mass, swarupa, velocity]

This is an IS-A assertion: "mass is [related to] velocity" in this sentence.

After: 15 triples (14 + 1 new)

#	New triple	Floor	Why
15	[mass, swarupa, velocity]	swarupa	copula pattern detected

What this step is in math terms

assertion-bandha reads the grammar structure and adds identity edges. In the boolean ring: swarupa is the multiplicative identity. If $G[\text{mass}][\text{velocity}][\text{swarupa}] = 1$, then mass and velocity are linked by identity in this question context.

Note: this swarupa edge is a question-local assertion, not a permanent graph edge. It says "in THIS question, mass and velocity are mentioned together as equals."

8. Step 3: avrti-refine-v2

What it does

Refines the question graph in two phases:

Phase 1 (11 sub-passes, run once): sandhi-kosha, sandhi-avastha, sandhi-bandhana, vyakarana-sparsha, shashthi-possession, vibhakti-shashthi, vishesa-instance, rashi-viveka, vishesa-bandhana, rashi-anuvada.

Phase 2 (fixpoint loop): sankhya-bandha-v2 $\rightarrow$ swarupa-anuvada $\rightarrow$ shunya-bandha-v2. Repeats until no new triples are added.

For our question

Most Phase 1 passes are no-ops (nothing to refine — no possessives, no rashi instances, no mithya words). The interesting work happens in Phase 2.

vyakarana-sparsha adds grammar annotations: - [kinetic-energy, kaala, vartamana-kaala] — present tense detected - [kinetic-energy, vachana, eka-vachana] — singular number detected

sankhya-bandha-v2: Already done by build-question-graph (mass=5, velocity=10 were bound at unit-recognition time). No new bindings.

swarupa-anuvada: No propagation needed (no chain of swarupa edges to follow).

shunya-bandha-v2: No unbound entities to mark as zero.

After: 17 triples (15 + 2 new)

#	New triple	Floor	Why
16	[kinetic-energy, kaala, vartamana-kaala]	kaala	grammar: present tense
17	[kinetic-energy, vachana, eka-vachana]	vachana	grammar: singular

What this step is in math terms

avrti-refine-v2 is a fixpoint operator: $f(G) = G$ when no new triples can be derived. It applies the ring operation $\oplus$ (OR / yukta) to accumulate triples monotonically. The graph can only grow, never shrink.

9. Steps 4-5: grade-split and count-chain

grade-split: What it does

Splits the flat list of triples into grades (sentences). The boundary markers are viraam (period) and dvandva (and).

For our question

Grade 0: triples 1-5 (mass is 5 kg) **Boundary:** triple 6 (and = dvandva = intra-grade boundary) **Grade 0 continued:** triples 7-11 (velocity is 10 m/s) **Boundary:** triple 12 (. = viraam = grade boundary) **Grade 1:** triples 13-14 (find kinetic energy) Plus: triples 15-17 (added by assertion-bandha and refine)

Grades are the graded ring structure: grade 0 = data, grade 1 = intent.

count-chain: What it does

Looks for counting patterns (how many X, what is the number of Y). For our question: no counting is needed, so this is a no-op.

After: still 17 triples (no change)

What this step is in math terms

grade-split implements the grading of the ring: $R = R_0 \oplus R_1 \oplus R_2 \oplus \dots$. Each grade is a sub-ring. The period (viraam) is the grade boundary — it resets the additive index.

10. Step 6: kosha-expand (PPR)

What it does

Personalized PageRank walk from each satya node. Discovers nearby concepts in the permanent graph that might be relevant.

For each node with a satya triple, PPR walks outgoing edges with decaying probability. Nodes above a threshold get added as [seed, kosha-janya, nearby-node].

For our question

Seeds: mass, velocity, kinetic-energy (the three satya nodes).

From mass, PPR discovers: momentum, kinetic-energy, gravitational-energy, force, kilogram, ... From velocity, PPR discovers: momentum, kinetic-energy, acceleration, displacement, ... From kinetic-energy, PPR discovers: potential-energy, mechanical-energy-varga, energy, momentum, velocity, mass, ...

After: 17 + N kosha-janya triples

The exact count depends on the threshold and top-k settings. Typical: ~20-40 new triples. These are all on the kosha-janya floor.

Example new triples:

Triple	Why
[mass, kosha-janya, momentum]	momentum is 1-hop from mass via yukta
[mass, kosha-janya, kinetic-energy]	kinetic-energy is 1-hop from mass via yukta
[velocity, kosha-janya, momentum]	momentum is 1-hop from velocity via yukta
[kinetic-energy, kosha-janya, potential-energy]	pratipaksha neighbor
[kinetic-energy, kosha-janya, velocity]	yukta neighbor

What this step is in math terms

PPR is a matrix operation on the adjacency matrix. Start with a seed vector $\mathbf{v}_0 = \mathbf{e}_{\text{mass}}$. At each step: $\mathbf{v}_{t+1} = \alpha \cdot A \cdot \mathbf{v}_t + (1 - \alpha) \cdot \mathbf{v}_0$. The stationary distribution tells us which nodes are "close" to the seed. This is a cross-floor walk: it reads edges from ALL floors (yukta, swarupa, phala, janya, ...) to find neighbors.

11. Step 7: detect-signals

What it does

Reads the question graph and detects what kind of answer is needed. There are 5 dispatch modes:

Mode	When
shunya	No intent, no data — empty question
anumana	"what is X" without numbers — definition lookup
viveka	"is X a Y" — yes/no classification
count	"how many X" — counting
derive	numbers + "find X" — computation (our case)

For our question

1. **extract-solve-for** scans for vidhi-kaala triples. Finds: [find, vidhi-kaala, solve-for]. The next satya node after "find" is kinetic-energy. So: solve-for = kinetic-energy.
2. **has-intent** = true (we have a vidhi-kaala edge).
3. **detect-shunya/viveka/anumana/count**: all return empty (we have numbers + intent).
4. **dispatch-mode** = "derive" (has-intent is true, no other mode matched first).

After: 17+N triples + 3 signal triples

Signal triple	What it means
[_signal, dispatch-mode, derive]	This is a computation question
[_signal, solve-for, kinetic-energy]	The target is kinetic-energy
[_signal, scope-entity, ""]	No scoping (e.g., "of the car")

What this step is in math terms

detect-signals is a classifier: it reads the boolean pattern of edges and maps to one of 5 categories. This is a morphism from the question graph to the dispatch lattice: $\text{Graph} \rightarrow \{\text{shunya}, \text{anumana}, \text{viveka}, \text{count}, \text{derive}\}$.

12. Step 8: dispatch-answer (derive path)

What it does

Reads dispatch-mode = "derive" and calls dispatch-derive, which:

1. **propagate-derive**: Reactive propagation — walks janya/phala edges from bound concepts to find matching mantras.
2. **match-mantra**: Finds the best mantra whose janya inputs match our bound concepts.
3. **execute-matched**: Evaluates the krama s-expression with the bound values.
4. **pramana-bandha + emit-reasoning**: Builds a proof graph (panchaavayava) and formats the answer.

Step 8a: match-mantra

Inputs: graph with sankhya triples (mass=5, velocity=10) and solve-for=kinetic-energy.

1. **mantra-select("kinetic-energy")**: Looks for mantras with phala → kinetic-energy. Finds: kinetic-energy-mantra.
2. **forward-match**: Checks if all janya inputs of kinetic-energy-mantra have values: - janya → mass: has sankhya 5. ✓ - janya → velocity: has sankhya 10. ✓ - All inputs present. Forward match succeeds.
3. Returns: [kinetic-energy-mantra, [(mass, 5.), (velocity, 10.)], "forward"]

Step 8b: execute-matched

The mantra has krama edges encoding: (half (multiplication mass (square velocity)))

execute-chain is a stack machine:

Step	Operation	Stack before	Stack after
1	push velocity=10	[]	[10]
2	square	[10]	[100]
3	push mass=5	[100]	[100, 5]
4	multiplication	[100, 5]	[500]
5	half	[500]	[250]

Result: 250

Step 8c: format answer

phala of kinetic-energy-mantra is kinetic-energy. Label = "kinetic-energy". Answer = "kinetic-energy = 250"

Step 8d: emit-reasoning (panchaavayava)

The proof is structured as 5 parts:

Part	Sanskrit	English	Content
1	pratijna	claim	we seek: kinetic-energy
2	hetu	reason	we know: kinetic-energy-mantra (velocity, mass → kinetic-energy)
3	udaharana	example	we see: velocity=10, mass=5
4	upanaya	application	kinetic-energy = half(mul(mass, square(velocity))) = half(mul(5, square(10)))
5	nigamana	conclusion	kinetic-energy = 250

Final output

"we have: mass=5. and velocity=10..we seek: kinetic-energy.we know: kinetic-energy-mantra (velocity, mass → kinetic-energy).we see: velocity=10., mass=5..we know: momentum-mantra (velocity, mass → momentum).we see: velocity=10., mass=5..we find: kinetic-energy = 250"

13. The Complete Triple History

Every triple that exists after each step. This is the full monotonic accumulation.

After build-question-graph (14 triples)

#	Subject	Edge	Object
1	mass	satya	mass
2	is	copula	is
3	5	asprista-sankhya	5.
4	mass	sankhya	5.
5	mass	matra	kilogram
6	and	dvandva	dvandva
7	velocity	satya	velocity
8	is	copula	is
9	10	asprista-sankhya	10.
10	velocity	sankhya	10.
11	velocity	matra	metre-per-second
12	.	viraam	.
13	find	vidhi-kaala	solve-for
14	kinetic-energy	satya	kinetic-energy

After assertion-bandha (+1 = 15)

After avrti-refine-v2 (+2 = 17)

16	kinetic-energy	kaala	vartamana-kaala
----	----------------	-------	-----------------

After kosha-expand (+N kosha-janya triples)

PPR triples are additive — they only help match-mantra find candidate formulas. They do not change the core binding logic.

After detect-signals (+3 signal triples)

S1	_signal	dispatch-mode	derive
S3	_signal	scope-entity	(empty)

Total: 20 + N triples

The graph only grew. No triple was ever removed. This is the monotonicity property of the boolean ring: $a \oplus b \geq a$ (OR never removes 1s).

14. Connecting to the Math Foundations

Sets

The graph is a set of nodes N with $|N| = 1604$. For our question, the active set is just ~ 12 nodes. The triples form a set of 3-tuples $(s, r, o) \in N \times R \times N$.

Relations

Each edge type (satya, sankhya, matra, ...) is a binary relation on the node set. For example, sankhya is a relation: $\{(mass, 5), (velocity, 10)\}$.

Operations

The pipeline applies operations on the triple set: - $\oplus$ (yukta / OR): adding a new triple = setting a 0 to 1
- $\otimes$ (kriya / AND): matching requires all conditions true simultaneously

Ring Structure

The question graph forms a boolean ring: - Additive identity (0) = shunya (no edge) - Multiplicative identity (1) = swarupa (identity edge) - $a \oplus a = a$ (adding the same triple twice = no change, idempotent)
- $a \otimes a = a$ (matching the same condition twice = same result)

Graded Ring

The sentences are grades: $R = R_0 \oplus R_1$ - R_0 = data grade: "mass is 5 kg and velocity is 10 m/s" - R_1 = intent grade: "find kinetic energy"

The period (viraam) is the grade boundary. Each grade has its own additive indexing.

Morphisms

The pipeline is a chain of morphisms (structure-preserving maps):

String $\xrightarrow{\text{BQG}}$ Triples $\xrightarrow{\text{refine}}$ Triples $\xrightarrow{\text{expand}}$ Triples $\xrightarrow{\text{detect}}$ Triples+Signals $\xrightarrow{\text{dispatch}}$ Answer

Each arrow preserves monotonicity (never removes triples) and respects the ring structure.

The 3D Tensor

For this question, we activated exactly these cells in $G \in \{0, 1\}^{N \times N \times R}$:

- $G[mass][*][satya] = 1$ (mass exists) - $G[mass][5][sankhya] = 1$ (mass = 5) - $G[mass][kilogram][matra] = 1$ (mass measured in kg) - $G[velocity][*][satya] = 1$ (velocity exists) - $G[velocity][10][sankhya] = 1$ (velocity = 10) - $G[velocity][m/s][matra] = 1$ (velocity measured in m/s) - $G[kinetic-energy][*][satya] = 1$ (kinetic-energy exists) - $G[find][*][vidhi-kaala] = 1$ (intent: solve) - $G[kinetic-energy-mantra][mass][janya] = 1$ (mass is input) - $G[kinetic-energy-mantra][velocity][janya] = 1$ (velocity is input) - $G[kinetic-energy-mantra][kinetic-energy][kinetic-energy] = 1$ (KE is output)

The pipeline walks these 1s to find the formula and compute the answer.

Composition (Panchaavayava)

The final answer is a 5-part proof: 1. pratijna (claim): we seek kinetic-energy 2. hetu (reason): kinetic-energy-mantra connects mass, velocity $\rightarrow$ kinetic-energy 3. udaharana (evidence): mass=5, velocity=10 4. upanaya (application): $\text{half}(\text{mul}(5, \text{square}(10))) = 250$ 5. nigamana (conclusion): kinetic-energy = 250
This is the composition Σ from the ring: the pipeline stages are the operations, the answer is the composed result.